RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS L2

Started on	Thursday, 27 August 2026, 10:12 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 11:55 AM
Time taken	34 days 1 hour
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int v,count=0;
    scanf("%d",&v);
    int coins[]={1000,500,100,50,20,10,5,2,1};
    int n=9;
    for(int i=0;i<n;i++)
    {
        if(v>=coins[i])
        {
            int coin=v/coins[i];
            count=count+coin;
            v=v%coins[i];
        }
    }
    printf("%d",count);
    return 0;

}
```

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS L2

Started on	Thursday, 8 October 2026, 11:12 AM
State	Finished
Completed on	Thursday, 8 October 2026, 11:19 AM
Time taken	7 mins 9 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort (ascending order)
int compare(const void *a, const void *b) {
    long long diff = (long long)*(int*)a - (long long)*(int*)b;
    if (diff > 0) return 1;
    if (diff < 0) return -1;
    return 0;
}

int main() {
    int n, m;

    // Read greed factors size and elements
    if (scanf("%d", &n) != 1) return 0;
    int g[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &g[i]);
    }

    // Read cookie sizes size and elements
    if (scanf("%d", &m) != 1) return 0;
    int s[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &s[i]);
    }

    // Sort both arrays in ascending order
    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    // Two-pointer approach
    int child = 0;
    int cookie = 0;

    while (child < n && cookie < m) {
        // If the smallest available cookie satisfies the current child
```

```
        // If the smallest available cookie satisfies the current child
        if (s[cookie] >= g[child]) {
            child++; // Move to the next child
        }
        cookie++; // Try next cookie regardless
    }

    // Output the total number of content children
    printf("%d\n", child);

    return 0;
}
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS L2

Started on	Thursday, 27 August 2026, 10:54 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 11:56 AM
Time taken	34 days 1 hour
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.

If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <stdlib.h>

// Safe comparison function for sorting long long values in descending order
int compare(const void *a, const void *b) {
    long long valA = *(const long long*)a;
    long long valB = *(const long long*)b;

    if (valB > valA) return 1;
    if (valB < valA) return -1;
    return 0;
}

int main() {
    int n;

    // Read the number of burgers
    if (scanf("%d", &n) != 1) return 0;

    // Using long long for calories array to safely process large numbers
    long long *calories = (long long *)malloc(n * sizeof(long long));
    if (calories == NULL) return 1;

    // Read the calories of each burger
    for (int i = 0; i < n; i++) {
        scanf("%lld", &calories[i]);
    }

```

```
    // Sort the calories array in descending order
    qsort(calories, n, sizeof(long long), compare);

    long long total_kilometers = 0;
    long long power_of_three = 1; // Represents 3^i, starting at 3^0 = 1

    // Calculate the minimum running distance
    for (int i = 0; i < n; i++) {
        total_kilometers += calories[i] * power_of_three;
        power_of_three *= n; // Move to the next power: 3^1, 3^2, ...
    }

    // Print the final result
    printf("%lld\n", total_kilometers);

    free(calories);
    return 0;
}
```

	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS

L2

Started on	Thursday, 17 September 2026, 10:35 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 11:57 AM
Time taken	13 days 1 hour
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort to sort in ascending order
int compare(const void *a, const void *b) {
    long long diff = (*(long long*)a - *(long long*)b);
    if (diff < 0) return -1;
    if (diff > 0) return 1;
    return 0;
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    long long *arr = (long long *)malloc(n * sizeof(long long));
    if (arr == NULL) return 1;

    // Read the array elements
    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    // Sort the array in ascending order: O(n log n)
    qsort(arr, n, sizeof(long long), compare);

    long long max_sum = 0;

    // Calculate the maximized sum: O(n)
    for (int i = 0; i < n; i++) {
        max_sum += arr[i] * i;
    }

    // Print the final result
    printf("%lld\n", max_sum);

    free(arr);
    return 0;
}

```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS

L2

Started on	Thursday, 17 September 2026, 10:37 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 11:57 AM
Time taken	13 days 1 hour
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for ascending order
int compare_asc(const void *a, const void *b) {
    long long diff = (*(long long*)a - *(long long*)b);
    if (diff < 0) return -1;
    if (diff > 0) return 1;
    return 0;
}

// Comparison function for descending order
int compare_desc(const void *a, const void *b) {
    long long diff = (*(long long*)b - *(long long*)a);
    if (diff < 0) return -1;
    if (diff > 0) return 1;
    return 0;
}

int main() {
    int n;

    // Read the size of the arrays
    if (scanf("%d", &n) != 1) return 0;

    long long *array_One = (long long *)malloc(n * sizeof(long long));
    long long *array_Two = (long long *)malloc(n * sizeof(long long));

    if (array_One == NULL || array_Two == NULL) return 1;

    // Read elements for array_One
    for (int i = 0; i < n; i++) {
        scanf("%lld", &array_One[i]);
    }

    // Read elements for array_Two
    for (int i = 0; i < n; i++) {
        scanf("%lld", &array_Two[i]);
    }

    // Sort array_One in ascending order
    qsort(array_One, n, sizeof(long long), compare_asc);

    // Sort array_Two in descending order
    qsort(array_Two, n, sizeof(long long), compare_desc);

    long long min_product_sum = 0;

    // Calculate the minimized sum of products
    for (int i = 0; i < n; i++) {
        min_product_sum += array_One[i] * array_Two[i];
    }
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course